

The Roof Model [DRAFT]

Open Specification



Table of Contents

Implementing the Model: A Guide to Hardware Purchase, Installation and Setup 2

1. Introduction.....3

2. Background 4

3. Objectives.....5

4. Logical Model of the Roof.....6

5. Core System Components of the Roof Model.....7

6. Hardware Specifications.....8

7. Software Requirements.....11

8. Roof Engine: Detailed Specification 12

9. RApps (Roof Native Apps) Development Guide 13

10. Hardware Purchase, Installation and Setup Guide.....14

11. Security Considerations 15

12. Community Contributions and Support 15

13. Future Developments and Roadmap 15

14. Open Source Licensing Information.....16

15. Appendix 17

Implementing the Model: A Guide to Hardware Purchase, Installation and Setup

For those looking to put this model into practical use, please consult our Hardware Purchase, Installation and Setup Guide found in Section 8.

1. Introduction

The Roof Model is a novel approach to personal cloud storage that reimagines how we store, access, and control our digital data. Unlike traditional cloud storage solutions that rely on centralised data centres owned by corporations, the Roof Model harnesses the capabilities of **single-board computers**, specifically the **Raspberry Pi Zero 2 W**, which can be powered via the USB port of a modem/router and connects to the network through Wi-Fi. This setup enables the creation of a decentralised, private cloud environment directly within the user's home. This model places the utmost importance on privacy, data sovereignty, and user autonomy.

Purpose and Core Principles

The purpose of the Roof Model is to empower individuals with full control over their personal data, mitigating concerns related to privacy, security, and dependence on third-party cloud providers. Its core principles include:

- **Decentralisation:** Reducing reliance on centralised cloud services by enabling personal devices to function as private cloud storage.
- **Privacy and Security:** Ensuring that users' data remains confidential and secure within their personal network.
- **User Access Control:** Granting users complete authority over their data, including how it's stored, accessed, and managed.
- **Accessibility:** Providing easy and secure access to data from anywhere, at any time, without sacrificing privacy or control.
- **Community Collaboration:** Encouraging open-source development and community contributions to continuously enhance and expand the model's capabilities.

2. Background

In response to the escalating concerns around data breaches and the erosion of privacy in the recent age, our proprietary Federated Database (FDB) initiative in work seeks to revolutionise data storage strategies, aiming to substantially enhance privacy and security.

A significant hurdle we faced during the development of FDB was maintaining continuous access to data, given the potential for accessibility issues when a device is turned off or disconnected from the network. To address this, we initially considered the application of blockchain decentralisation technology, which inherently distributes data across numerous locations. However, this approach appeared to compromise our core objective of safeguarding user data privacy and autonomy, given the broad dispersal of data it involved.

In our quest for a solution that aligns with our commitment to user privacy and data control, we introduced the "Roof Model." This model is inspired by the common household modem, maintains a constant internet connection, ensuring availability 24/7 except during outages. This concept represents a significant evolution from traditional cloud storage, which, despite offering convenience, often necessitates a trade-off in terms of control and privacy, rendering personal information susceptible to unauthorised access.

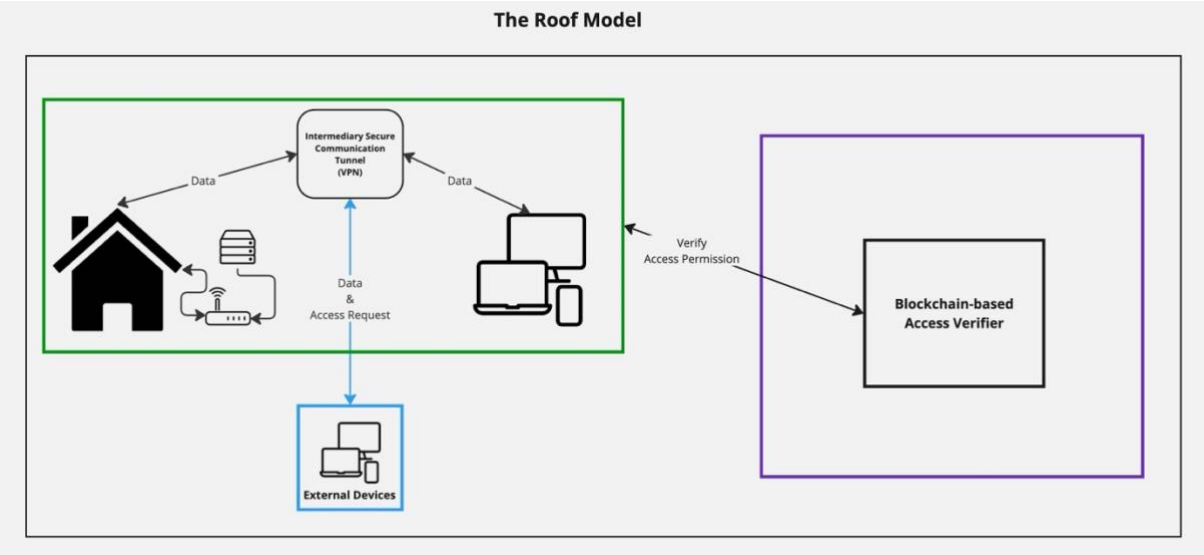
The Roof Model signifies an advanced stage in this evolution, returning control to the users and establishing a foundation for a future where personal data storage is both private and centred around the user's preferences and control.

3. Objectives

- **Enhancing Data Privacy and Sovereignty**
  One of the primary objectives of the Roof Model is to enhance the privacy and sovereignty of user data. By decentralising storage and keeping data within the user's personal network, the Roof Model ensures that users retain ownership and control over their information, free from the oversight of external entities.
- **Promoting User-Controlled Data Management**
  The Roof Model promotes an environment where users are in complete control of their data management. This includes deciding who has access to their data, where it is stored, and how it is encrypted. This level of control is pivotal in establishing trust and confidence in digital storage solutions.
- **Fostering an Open, Community-Driven Ecosystem**
  The Roof Model is not just a technical solution but a community-driven initiative. It aims to foster an ecosystem where developers, users, and enthusiasts can collaborate to improve, customise, and innovate, ensuring the model remains adaptable, secure, and aligned with the needs of its users. Through open-source development and community support, the Roof Model encourages a collective approach to solving the challenges of personal data storage.

4. Logical Model of the Roof

The logical Roof Model is designed as a decentralised, secure personal cloud storage system that leverages the capabilities of single-board computers like the Raspberry Pi Zero 2 W, integrating advanced blockchain technology for access control and VPN tunnels for encrypted communication. This model prioritises user privacy, data sovereignty, and broad accessibility, allowing secure access from authorised devices without relying on centralised data centres. By employing Self-Sovereign Identity (SSI) for managing digital identities and permissions, the Roof Model ensures that users retain complete control over their data, making it accessible only to those with verified permissions. Refer to the model diagram below.



5. Core System Components of the Roof Model

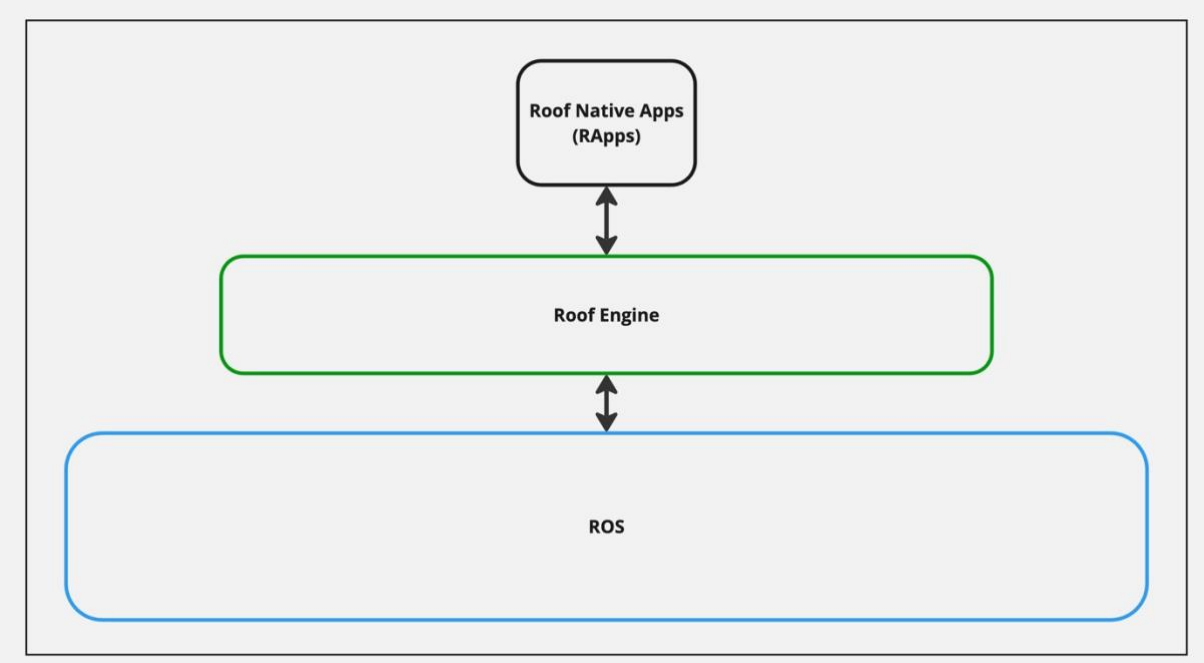
The Roof Model is built on a foundation of three key components, each designed to optimise the performance, security, and functionality of personal cloud storage systems. These are the Roof Engine, the Federated Database (FDB), and Roof Native Apps (RApps).

Role and Functionality of Each Component

Roof Engine: The central processing unit of the Roof Model, the Roof Engine, runs atop the Raspberry Pi Operating System (ROS) and is responsible for managing the system's core operations. It allocates memory, manages resources, ensures that the system runs smoothly and efficiently, and hosts a file storage engine. This integrated file storage engine is essential for organising, storing, and retrieving user data effectively within the personal cloud storage solution. The Engine is designed to be lightweight yet powerful, capable of supporting a wide range of functionalities through plugins, including advanced data management and security features. This makes the Roof Engine a comprehensive hub for both system operations and data handling, providing a robust foundation for the Roof Model's decentralised personal cloud storage capabilities.

Roof Native Apps (RApps): RApps extend the functionality of the Roof Model, allowing developers to create and users to install additional features that enhance their personal cloud. From security enhancements to custom user interfaces, RApps can be compiled as native libraries and integrated directly into the Roof Engine, providing a flexible and powerful way to customise and improve the Roof Model.

Refer to the system diagram below.



6. Hardware Specifications

Hardware for Implementation: Raspberry Pi Zero 2 W

In the development of the Roof Model, our goal was to leverage the built-in file-sharing capabilities of modern modems/routers through the USB port, a feature commonly supported by default. We aimed to combine this with port forwarding to enable remote access, a critical component for the functionality of our Roof model. However, we encountered several challenges that ultimately led us to select the Raspberry Pi Zero 2 W as our preferred hardware.

- **Challenge 1. The Challenge with Modems and Routers**
  Most standard home modems and routers lack the ability to perform self-port forwarding due to security, an essential feature for the Roof model's remote access capabilities.
- **Challenge 2. Lack of USB's Hardware Capability**
  We experimented with using USB flash drives as makeshift file servers, hoping to exploit their simplicity and ubiquity. Unfortunately, this approach was hindered by the drives' lack of necessary hardware components, such as network interface controllers, rendering them unsuitable for the task.
- **Challenge 3. Legal and Technical Barriers**
  We then explored the possibility of modifying modem software to enable self-port forwarding. This solution, however, presented potential legal and copyright issues, alongside the risk of disrupting the modem's core functionalities. Additionally, it takes a considerable amount of effort and time to reverse engineer the modem's software protocol.
- **Challenge 4. Cost Concerns with Advanced Hardware**
  Considering advanced routers and Network Attached Storage (NAS) solutions was a logical step, given their out-of-the-box support for file sharing and server capabilities. Yet, the high cost associated with these devices posed a significant barrier, making them inaccessible to a broad user base.

Our journey led us to reassess our hardware choice, initially favouring the **TP-Link Archer Wireless Dual Band Travel Router AC750** for its concurrent support for self-port forwarding and USB file sharing. Nevertheless, the router's limited customisation options and potential security vulnerabilities related to its built-in SMB & FTP functionalities prompted us to seek an alternative.

Ultimately, we identified the **Raspberry Pi Zero 2 W** as the ideal hardware for the Roof Model. This decision was based on several key factors:

- **Affordability:** The Raspberry Pi Zero 2 W's cost-effectiveness makes it accessible to a wide audience, aligning with our goal of democratising personal cloud storage.
- **Customisation and Control:** Unlike many off-the-shelf routers, the Raspberry Pi Zero 2 W offers extensive customisation capabilities, allowing for the development of a secure and personalised cloud storage solution.
- **Community and Support:** The robust Raspberry Pi community provides invaluable resources, tools, and support, which are crucial for the iterative development and troubleshooting process.

- **Efficiency and Compactness:** Its low power consumption and small size perfectly match the Roof Model's emphasis on efficiency and minimalism.

Hardware Peripherals

To ensure the Roof Model operates efficiently and meets the performance standards expected for decentralised personal cloud storage, specific hardware peripherals are recommended. These peripherals not only facilitate the basic operation of the system but also enhance its functionality, allowing users to fully leverage the capabilities of their personal cloud storage solution. Below is a detailed list of the required hardware peripherals and their purposes:

- **Micro USB (Male) to USB - A (Male) Cable:** Essential for powering the device, this cable connects the Raspberry Pi Zero 2 W to a power source (Modem/Router). Given the device's minimal power requirements, this setup ensures a stable operation without the need for a bulky power supply.
- **Micro USB (Male) to USB - A (Female) On-The-Go (OTG) Adapter:** This adapter is crucial for connecting USB storage drives to the primary device, Raspberry Pi Zero 2 W. It enables the device to read from and write to the USB storage, acting as the main interface for data storage and retrieval.
- **MicroSD Card (32GB):** A 32GB MicroSD card is recommended to achieve maximum efficiency and performance. This card serves as the primary boot and operating system storage, holding the Raspberry Pi OS and the Roof Model software.
- **USB - A Flash Drive:** Acts as the main storage medium for the Roof Model, housing user data, databases, and any other information managed by the system. The USB flash drive is connected via the OTG adapter, expanding the system's storage capabilities beyond the MicroSD card.

Expansion Consideration for Storage Devices

As the needs of users grow, so does the requirement for expanded storage within the Roof Model ecosystem. To accommodate this growth and ensure the system remains scalable, several options for storage expansion are recommended. These options not only enhance the storage capacity but also introduce versatility in the types of storage devices that can be utilised with the system.

- **USB Hub:** A powered USB hub can significantly expand the number of devices that can be connected to the primary single-board computer, such as the Raspberry Pi Zero 2 W. This is particularly useful for attaching multiple USB storage devices without overloading the device's power supply.
- **NAS Storage Block:** For users requiring substantial data storage or wishing to create a more robust network-attached storage (NAS) system, incorporating a NAS storage block into the Roof Model can provide a centralised storage solution accessible by multiple devices within the network.
- **USB to SATA Adapter:** To enable the connection of SATA-capable HDDs or SSDs, which offer higher storage capacities and faster data access speeds than typical USB flash drives, a USB to SATA adapter can be used. This allows for the use of more traditional hard drives or solid-state drives as part of the Roof Model's storage solution.

Consideration of the Raspberry Pi Zero 2 W's RAM Capacity

The Raspberry Pi Zero 2 W, while an excellent choice for the Roof Model due to its size, cost, and efficiency, comes with a notable limitation: its 512MB of RAM. This constraint necessitates careful consideration and strategic planning to ensure the system runs efficiently, especially when managing personal cloud storage solutions that may require significant memory resources.

- **Optimising Memory Management**
  The initial approach to addressing this limitation revolves around optimising memory usage to ensure the Raspberry Pi Zero 2 W operates within its capabilities without compromising performance. Key strategies include:
  - **Efficient Software Design:** Developing the Roof Engine and associated components with memory efficiency in mind, minimising unnecessary memory consumption.
  - **Utilising Swap Space:** Implementing swap space on the MicroSD card or an attached USB drive can help alleviate memory pressure by providing additional virtual memory. While swap can help manage memory constraints, it's slower than RAM and should be used judiciously.
  - **Selective Service Running:** Only essential services and applications should be running to conserve memory. This might involve disabling or removing background processes that are not critical to the Roof Model's operation.
  - **Leveraging Lightweight Alternatives:** Where possible, use lightweight versions of software and tools that fulfill the necessary functions without the heavy memory footprint.
- **Future Considerations for Upgrades**
  While optimisation strategies can significantly mitigate the impact of limited RAM, there may come a point where the demands of the Roof Model exceed what can be reasonably managed with 512MB. At this juncture, considering an upgrade to a higher-capable Raspberry Pi model becomes pertinent. Upgrade considerations should include: **Evaluating Performance Needs, Cost vs. Benefit Analysis,** and **Seamless Transition Planning**.

7. Software Requirements

For the Roof Model to function optimally, specific software requirements must be met. These requirements ensure that the system runs efficiently, securely, and remains scalable. Below are the detailed specifications for the operating system, as well as the essential software and libraries needed.

Operating System Specifications

Raspberry Pi OS Lite: The recommended operating system for single-board computers (SBCs) within the Roof Model ecosystem is Raspberry Pi OS Lite. This choice is due to its lightweight nature, which is particularly suited for the Raspberry Pi Zero 2 W's hardware specifications. The OS Lite version is stripped of unnecessary components, ensuring that the system resources are primarily allocated to the core functionalities of the Roof Model.

- **Version:** It is advised to use the latest stable release of Raspberry Pi OS Lite to benefit from the most recent security patches, performance improvements, and software updates.

Required Software and Libraries

To enable the core functionalities of the Roof Model, including data management, network communication, and user interface, the following software and libraries are essential:

- **Roof Engine Dependencies:** The Roof Engine may require specific libraries for operation, depending on the programming language and framework used. Common dependencies might include database drivers, web server software (e.g., Nginx for serving web content), and cryptographic libraries for secure data handling.
- **VPN Software:** To establish secure VPN tunnels for remote access, VPN client and server software must be installed and configured.
- **Blockchain Libraries:** If implementing decentralized access control via blockchain technology, relevant blockchain client libraries and tools need to be integrated. This enables the Roof Model to interact with blockchain networks for managing digital identities and access permissions.
- **Development Tools:** For developers contributing to the Roof Model, a standard set of development tools including compilers, debuggers, and code editors, specific to the development environment, should be installed. Additionally, version control software like Git is essential for managing code contributions and collaboration.

8. Roof Engine: Detailed Specification

The Roof Engine is the core component of the Roof Model, designed to power decentralised personal cloud storage solutions with a focus on privacy, security, and efficiency. Below are detailed specifications covering its architecture, memory management strategies, and security protocols.

Architecture and Design Philosophy

- **Modular Design:** The Roof Engine is built on a modular architecture, allowing for easy expansion and customisation. This design philosophy supports the integration of plugins (Roof Native Apps - RApps) and facilitates updates and maintenance without disrupting the overall system functionality.
- **Lightweight and Efficient:** Optimised for use on single-board computers like the Raspberry Pi Zero 2 W, the Roof Engine prioritises efficiency. Its design minimises resource consumption while maintaining high performance, making it suitable for devices with limited processing power and memory.
- **User-Centric and Secure:** At its core, the Roof Engine is developed with the end-user's privacy and security in mind. It employs end-to-end encryption for data storage and transmission, ensuring that user data remains confidential and protected.

Memory Management and Performance Optimisation

- **Dynamic Memory Allocation:** The Roof Engine implements advanced memory management techniques to optimise the use of the limited RAM available on devices like the Raspberry Pi Zero 2 W. It dynamically allocates and deallocates memory based on current system demands, ensuring smooth operation without sacrificing performance.
- **Swap Space Utilisation:** To extend the available memory, the Roof Engine uses swap space efficiently, creating additional virtual memory on the MicroSD card or an attached USB drive. This approach is carefully balanced to minimise reliance on swap, which can be slower than RAM, focusing instead on optimising in-memory operations.
- **Performance Tuning:** Regular performance profiling and tuning are integral to the Roof Engine's development cycle. This ensures that the engine runs optimally under various conditions, with adjustments made as needed to address any bottlenecks or inefficiencies identified.

Security Features and Protocols

- **Encryption:** The Roof Engine incorporates strong encryption standards for both data at rest and in transit. It uses industry-standard algorithms to encrypt user data, ensuring that sensitive information is securely stored and communicated.
- **Access Control:** Leveraging decentralised technologies like blockchain for access management, the Roof Engine provides a secure and transparent mechanism for managing user permissions. This allows for the implementation of Self-Sovereign Identity (SSI) principles, giving users control over their digital identities and access rights.
- **VPN Support:** For secure remote access, the Roof Engine integrates VPN technologies, creating encrypted tunnels for data transmission. This ensures that users can access their personal cloud storage securely from anywhere, without exposing their data to potential network threats**.**

- **Regular Security Audits:** The development process includes regular security audits and updates to address vulnerabilities, adhere to best practices, and respond to emerging threats. This proactive approach to security helps protect the Roof Model and its users from potential risks.

9. RApps (Roof Native Apps) Development Guide

Creating Plugins for the Roof
API Documentation and Developer Tools
Best Practices for Development and Security

10. Hardware Purchase, Installation and Setup Guide

Hardware Purchasing Guideline

Setting up the Roof begins with acquiring the necessary hardware and peripherals. This guide provides a comprehensive overview to assist you in purchasing, installing, and setting up your Roof system, ensuring a smooth and successful deployment.

When purchasing hardware for the Roof, prioritise compatibility and quality to ensure reliable performance. Consider buying from reputable suppliers or official channels to guarantee authenticity and receive support if needed. Keep the following in mind:

- **Compatibility:** Ensure all peripherals are compatible with the Raspberry Pi Zero 2 W.
- **Quality:** Opt for high-quality components that offer durability and reliable performance.
- **Warranty:** Check for warranty options, especially for the Raspberry Pi Zero 2 W and storage devices, to protect your investment.

To set up the Roof, you will need the following essential hardware and peripherals:

- **Raspberry Pi Zero 2 W:** The core component of the Roof Model, selected for its efficiency, cost-effectiveness, and support for Linux-based operating systems.
- **Micro USB (Male) to USB-A (Male) Cable:** Used to power the Raspberry Pi Zero 2 W from a suitable USB power source.
- **Micro USB (Male) to USB-A (Female) On-The-Go (OTG) Adapter:** Essential for connecting USB peripherals such as storage devices to the Raspberry Pi Zero 2 W.
- **MicroSD Card (32GB):** Serves as the primary storage for the operating system and software. A high-speed class 10 card is recommended for optimal performance.
- **USB-A Flash Drive:** Acts as the main storage for your data. The size depends on your storage needs; however, selecting a high-speed drive is advisable for faster data access.
- **A laptop or a desktop PC with SD Card reader:** Necessary for initial setup of the Raspberry Pi OS on the MicroSD card. If your computer lacks an SD card reader, a standalone USB MicroSD Card Reader will be required.

Optional Peripherals

For enhanced functionality and ease of setup, the following optional peripherals may be considered:

- **Mini HDMI (Male) to HDMI (Male) Cable:** Useful for direct debugging or setting up the Raspberry Pi Zero 2 W with a monitor, facilitating an easier initial configuration.
- **Raspberry Pi Zero 2 W Case:** Protects the Raspberry Pi Zero 2 W from physical damage and can help with heat dissipation, extending the longevity of your device.

Installation and Setup

11. Security Considerations

Vulnerability and Risk Assessment of The Roof Model

12. Community Contributions and Support

Guidelines for Contributing to the Roof Project
Resources for Getting Support and Collaborating with Others

13. Future Developments and Roadmap

Planned Features and Enhancements

14. Open Source Licensing Information

The Roof Model is dedicated to fostering innovation, collaboration, and transparency in crafting decentralised personal cloud storage solutions. In line with this mission, we have opted for the **GNU General Public License version 3 (GPLv3)** for our open-source licensing framework. This choice underscores our commitment to a community-driven ecosystem, encouraging developers to **actively contribute to and derive benefits** from the project, while ensuring the Roof Model **maintains its integrity as a standardised, cohesive platform**.

How GPLv3 Applies to the Roof Project

Under the Roof Model, GPLv3 is applicable to all core components and contributions made directly to those components. This encompasses the Roof Engine and all other essential parts of the system. Contributors who modify or enhance these components are mandated to share their changes under GPLv3, guaranteeing that advancements to the core system remain open and accessible to the broader community.

GPLv3's strong copyleft clause ensures that any derivative works remain free and open, promoting an environment where improvements and innovations benefit all. This approach aligns with our vision of a transparent, collaborative development process.

However, GPLv3 also provides mechanisms for interacting with proprietary systems, allowing the development of separate works that communicate with the Roof Model without becoming subject to the same licensing constraints, provided they are truly separate and not derivative. This nuance encourages broader participation in the Roof Model ecosystem, enabling businesses and developers to leverage the model for proprietary applications while contributing back to its foundational, open-source core.

15. Appendix

- **Glossary of Terms**
- **Troubleshooting Common Issues**
- **Links to Additional Resources**